darkmpc, Milestone 5 walkthrough

Limit prices, a leakage audit, and real machines

Tave La Rocque

6 November 2026

Milestone 5a: an auction with prices

What changes when orders carry limits

In Stage 1 every order traded at the public midpoint, so the only question per symbol was how much. Stage 2 lets each order name a limit: a buyer will pay at most its limit, a seller will take at least its limit. Now the auction must also choose a price, and different prices let different orders in. The rule used here is the one most periodic auctions use. For each possible price p, demand $D(p)$ is the total size of buys willing to pay p, and supply $S(p)$ is the total size of sells willing to accept it. The volume that can trade at p is the smaller of the two. The auction picks the price with the most volume. If several tie, it picks the one where the leftover imbalance $|D - S|$ is smallest. If they still tie, it takes the lowest. Those first two steps are the first two criteria Cboe Europe publishes for its Periodic Auctions; the third is my simplification, because Cboe's later steps need a public reference price, which would have to be a secret lookup when the symbol is hidden.

The plain version of this rule is easy: try every limit price that appears, compute D and S, keep the best. The MPC version cannot "try every price and keep the best" cheaply, because every comparison on shares costs rounds, and the servers must do the same work whatever the data. So the real design question was how to find the best price without a search.

The shape of demand and supply

The answer comes from the shape of the two curves. As the price rises, buyers drop out, so demand can only fall; sellers join, so supply can only rise. Picture two staircases, one stepping down and one stepping up. They cross once. Before the crossing, supply is the short side, so the volume is S, which grows as you move right. After it, demand is the short side, so the volume is D, which shrinks. The best price must sit right at the crossing.

That leaves exactly two contenders: the first price at or after the crossing, and the best price just before it. The first is unambiguous. The second hides a trap, and I fell into it. I assumed the best price before the crossing was simply the last one, the price immediately to the left. The property test, which compares the shortcut with

the brute-force search on random books, found a two-order counterexample within seconds: a buy of 13 at 101 and a sell of 1 at 95. At both prices demand is 13 and supply is 1, because nothing enters or leaves between them. A buy only drops out *above* its limit and a sell only joins *at* its limit. The two prices tie on everything, and the lowest-price rule picks 95, not 101. The fix is to take the first price of the flat stretch that ends just before the crossing. The lesson is the reason the reference exists: the mistake was in the mechanism, not the cryptography, and a plain program with an oracle caught it before a single share was involved.

Doing it on shares

Sort the orders by symbol and then limit. Orders with the same symbol and the same limit form a *run*. Along a symbol, the runs appear in increasing price order, which is exactly the order in which to walk the two staircases.

Demand at an order's own limit is the buys from its run to the end of the symbol. Supply is the sells from the start of the symbol to the end of its run. Both are sums over the symbol corrected by sums over the run, and sums over segments are what the scans from Milestone 3 compute. So every order learns, on shares, the (D, S) of its own price.

Finding the crossing is then local. Let $c = [S \geq D]$ at each position. It is 0 and then 1 along a symbol, never back, so the first 1 is where c differs from its predecessor: one multiplication, no search. Finding the flat stretch uses the same shape. Because D only falls and S only rises, the quantity $(D_{\text{prev}} - D) + (S - S_{\text{prev}})$ is never negative, and it is zero exactly when nothing changed. One comparison with zero detects a change. Wherever a change happens a new block starts, and the block holding the last position before the crossing is the flat stretch; its first position is the second candidate.

Each symbol then sums its two candidates' volume, surplus and price into every position, compares them, and selects the clearing price. Every step has the same cost whatever the books look like, which is what makes it safe to run on secrets.

Allocation without a second sort

The draft design planned a second sort, because allocation needs a different order: sells by lowest limit first, buys by highest limit first, then arrival. Writing the code showed that the first sort already gives both. For sells, ascending limit then arrival is the sorted order itself. For a buy, the eligible buys ahead of it are those at higher limits, which are the later runs of its symbol, plus those earlier in its own run. That is once more a symbol sum minus a run sum. So allocation needs scans, not a sort, and the clamp formula from Stage 1, $\text{fill} = \min(\text{size}, \max(0, V^* - \text{ahead}))$, finishes the job.

A related saving applies to both stages. A scan over n positions takes about $\log_2 n$ rounds, and the auctions run several scans side by side: forward and backward, over symbols and over runs. Laying them end to end in one long sequence, with each piece starting a new segment so no sum leaks from one piece into the next, runs them all in one scan of a few more rounds. That cut Stage 1 from 316 to 280 rounds at 1,024 orders.

What it costs, and what a trader learns

Measured on simulated networks, Stage 2 sends about two and a half times Stage 1’s data, but on a 40 ms network it takes only 1.1 to 1.3 times as long, because there time is set by rounds, and Stage 2 has almost the same round count. Its sort is longer, but it has no elimination passes.

A filled trader must learn the price, or the trade cannot settle, so each order’s output is its fill and, if filled, the price. That reveals something the cryptography cannot hide: the price is always some order’s limit, so a filled trader whose limit differs learns that someone named exactly that price. This leak comes from how the auction is defined, not from the protocol, and the write-up says so.

Three planted bugs guard the new code: dropping the flat-stretch candidate, letting a sell trade only strictly below the price, and allocating buys by arrival alone. Each one breaks the differential tests, which is the evidence that the tests can see those mistakes.

Milestone 5b: proving that a server learns nothing

What “learns nothing” means

A server in darkmpc sees a great deal. It holds shares of every order, it receives millions of numbers from the other two servers during an auction, and every so often the servers open a value and all three see it in the clear. The claim that it learns nothing about the orders cannot mean that it sees nothing. It has to mean that everything it sees is useless for telling one batch of orders from another.

Cryptographers make this precise with a *simulator*. Imagine a program that is told only the public facts, the batch size and the configuration, and never sees an order. If that program can produce a fake transcript that looks exactly like what the real server would see, then the real transcript cannot contain information about the orders. Anything the server could compute from its real view, it could have computed from the fake one, which was made without the orders. The whole proof is a recipe for that program, plus an argument that its output is indistinguishable from reality.

Why the recipe is short

The recipe is short because the protocol is built from only four kinds of interaction, and each has a simple reason to be harmless.

The first is the server’s own shares of the inputs. A trader splits each number into three random-looking pieces that add up to it, and each server holds two. Any two of the three pieces are just two uniformly random numbers, whatever the order was, so the simulator draws two random numbers.

The second is the message that arrives during a multiplication. It carries a mask built from a pseudorandom stream that the receiving server has no key for. Without the key, that mask is as good as fresh random noise, and noise added to anything is uniform noise. The simulator sends random words. This is the step that leans on cryptography: if AES could be told apart from randomness, the argument would break, and the proof’s first move, the *hybrid* step, replaces that one stream with true randomness and pays exactly the PRF’s distinguishing advantage for it.

The third is the shuffle. Each of its three steps is run by a pair of servers and hides its piece of the permutation from the third. When the hidden piece is this server’s own blind spot, the two numbers it receives are masked by fresh randomness from that same keyless stream, so they are uniform too. In the other two steps it only sends.

The fourth, and the only place a server sees a value in the clear, is the opening of a shuffled permutation. The servers need to apply a secret sorting permutation ρ, so they first compose it with the random shuffle π and only then open it. Our server knows two of the three pieces of π but not the third, and composing anything with a uniformly random permutation it does not know gives a uniformly random permutation. So the opened value is a random permutation, whatever the data was, and the simulator draws one. The server then applies that public permutation to its shares. It has learned a random shuffle of the data’s order, which is to say nothing.

Put the four together in the order the protocol runs them, and the simulator has a rule for every message. Because each rule produces the right distribution *given everything before it*, the whole fake transcript has the right distribution. That chaining step is what turns four local facts into a theorem about an entire auction.

What the tests add

A proof is about the protocol as written on paper. The tests ask whether the code is that protocol. The proof needs two facts about the code, and each has a test.

The first fact is that the *shape* of the traffic, how many messages and how long, depends only on the public facts. If the code ever branched on a secret, say by skipping work for an empty book, the message pattern itself would leak. The shape test runs two very different batches and requires every server's traffic to match exactly, message for message.

The second fact is that every opening is either a random permutation or a fixed constant. The opening test runs each batch hundreds of times and checks, opening by opening, that the distribution of what was opened does not depend on the batch.

A test is only worth something if it can fail, so two leaks are planted. One forgets to shuffle before opening, which publishes the order of the data. The other leaves behind a stray "print the clearing price" opening, a mistake real developers make while debugging. Neither changes a single fill or price, so every correctness test in the project passes with them switched on. The audit catches both, at the strongest signal the statistic can give. That is the point of the exercise: a class of bug that no amount of output checking can see, caught by checking the view instead.

The honest limits

The proof covers servers that follow the protocol. A server that actively cheats could try to make an opening depend on a secret, for example by tampering with a shuffle and watching whether the result is still a valid permutation. Milestone 4 built the defences against that. Proving that they work, with simulation against a cheating server and the two ring extensions argued in full, is the main proof of the final write-up. Timing and memory side channels are outside the proof altogether. And nothing here hides what the auction itself must reveal to traders, which the threat model lists plainly.

Milestone 5c: measuring on real machines

Why simulation was not enough

Until now every timing came from one computer pretending to be three. The three servers ran as threads or processes, and a small model delayed each message by a chosen latency and a chosen bandwidth. That is good for comparing two versions of the code, and it caught real mistakes, like the “hidden round” in the checksum optimisation. But a model only knows what you tell it. It does not know how a real kernel schedules threads, how TCP behaves over a 68 millisecond link, or what a cloud VM’s network actually delivers. A benchmark that ends up on a resume has to come from machines that were really apart.

So the last measurement of Milestone 5 puts each server on its own Google Cloud VM. A script on the laptop creates the three machines, copies the committed code to them, builds it there, measures the network it got, and then runs every configuration three times. The laptop plays the traders: it deals each batch into shares and uploads one share file to each server. After each auction it downloads the three output files, reconstructs every fill, and compares them with the plain reference matcher. A run that disagrees is never recorded. That check is what makes the table trustworthy: every number in it comes from an auction that provably produced the right answer.

The first lesson was about quotas, not cryptography

The first attempt failed before any MPC ran. Free-trial projects may use at most 8 vCPUs of Google’s newest machine family per region, and the plan asked for 12. The fix was smaller machines with one physical core each. That makes these numbers a floor for the hardware: every server had less compute than a typical laptop. The script also had a real bug, exposed by the failure. When one of three VMs could not be created, the other two kept running and billing. It now deletes whatever it created if any part fails, a property worth having in any script that rents machines by the second. The second region tried for the WAN had simply run out of that machine type, a reminder that “the cloud” is a set of real buildings with real stock.

What the numbers say

On a LAN, with all three servers in one data centre and 0.2 milliseconds between them, Stage 1 handles 1,000 orders in 0.055 seconds in semi-honest mode and 100,000 orders in 4.3 seconds. With malicious security, the version that catches a cheating server, 1,000 orders take 0.95 seconds and 10,000 take 9.9, a steady thousand orders per second. The roughly twentyfold cost of malicious security is almost all data: every multiplication is checked against a bucket of random triples, and producing those triples is most of the traffic.

The WAN tells a different story. With servers in Oregon, Iowa and Northern Virginia, every round of communication costs about 25 milliseconds, a bit under half the 68 milliseconds it takes a message to cross the country and come back. The protocol needs a few hundred rounds whatever the batch size, so even a 100-order auction takes 6 seconds. The arithmetic is trivial; the time goes into waiting for the speed of

light. That is why the round count, which grows only logarithmically with the batch, was the number worth optimising throughout this project. It is also why large batches pay: the semi-honest WAN throughput grows from 17 orders per second at 100 orders to over 4,000 at 100,000, because the same few hundred rounds now carry a thousand times the work. At 100,000 orders the second limit appears, bandwidth: 2 gigabytes per server for Stage 2 take most of a minute to cross a 0.7 Gbit/s link.

The comparison, and its limits

The one published system to compare against is the 2018 three-party dark pool by Cartlidge, Smart and Talibi Alaoui. For 1,000 orders with malicious security it reports 0.9 seconds of online time plus 4.1 seconds of preprocessing, about 5 seconds end to end. darkmpc does the same size in 0.95 seconds with no preprocessing at all, about five times faster end to end, using 566 rounds where they use 4,509. And it does more work while doing so: it hides every order’s symbol among 8,192, it validates traders’ inputs, and it runs minimum-fill elimination, none of which their volume match does.

A fair account also lists what favours darkmpc. Its LAN had half the latency of theirs. Its cores are newer, though there are fewer of them. And it is purpose-built, where theirs ran in a general framework. Those differences cannot be separated from these numbers, so the honest statement is about the two systems as measured, not about one algorithm beating another. On a WAN there is nothing to compare: the 2018 paper never measured one. darkmpc’s 17 seconds for an actively secure 1,000-order auction across a continent is a new data point, and the reason it is that low is the round count.

Appendix A: the leakage proof

This document states exactly what one server learns from running either auction, proves that statement for the semi-honest protocol, and describes the tests that go with the proof. The tests are sanity checks: they catch specific implementation mistakes, and two planted leaks show that they do. The proof is the actual argument.

A. 1. What is claimed

Setting. Three servers P_0, P_1, P_2, indices modulo 3. At most one server, P_i, is corrupt. In this section it follows the protocol and tries to learn from what it sees (semi-honest). Traders are honest here; malicious traders are handled by validation (Section 5).

Public parameters. $\mathsf{pub} = (n, \text{symbol bits}, \text{passes}, \text{stage}, \text{mode})$: the batch size and the configuration. Nothing about any order.

The view of P_i. Everything it holds at the end of a run:

1. its shares of every input,
2. every message it receives, in order,
3. every value it learns from an opening.

Servers receive no output. Fills and prices go to traders as shares, so no server sends anything that depends on them back to another server.

Theorem. Assume F (AES-128 in counter mode, `crates/mpc/src/prf.rs`) is a pseudorandom function and the traders' share randomness is uniform. For each i there is a simulator $\mathcal{S}_i$ which, given only pub, outputs a view whose distribution no efficient distinguisher can tell from the real view of P_i, on any batch of n valid orders, for either stage. The distinguishing advantage is at most

$$\mathrm{Adv}_F^{\mathrm{prf}} \;+\; S \cdot \frac{n^2}{2^{65}},$$

where S is the number of shuffles in the run (one per sort bit plus the unsort) and the second term is the modulo bias of the permutation sampler.

A. 2. Notation

A value x in $\mathbb{Z}_{2^{64}}$ is split as $x = x_0 + x_1 + x_2$, and P_j holds the pair (x_j, x_{j+1}). So P_i is missing exactly x_{i+2}. Boolean sharing is the same with XOR.

Each pair of servers shares one PRF key. Key k_j is held by P_j and P_{j-1}, so P_i holds k_i and k_{i+1}, and the one key it does not hold is

$$k_{i+2}, \quad \text{shared by } P_{i+1} \text{ and } P_{i+2}.$$

Each key drives one stream of words, and the code consumes every stream sequentially: no word is ever used twice.

A. 3. The proof, step by step

Hybrid H_0 is the real view of P_i.

Hybrid H_1 replaces the stream of k_{i+2} by truly uniform words.

$$|\Pr[D(H_0) = 1] - \Pr[D(H_1) = 1]| \le \mathrm{Adv}_F^{\mathrm{prf}}.$$

The reduction plays P_{i+1} and P_{i+2} honestly, using its oracle wherever they would evaluate $F(k_{i+2}, \cdot)$, and plays P_i with P_i's own keys. With the real PRF it produces H_0, and with a random function it produces H_1.

Everything below is in H_1.

Step 1: input shares. A trader shares x by drawing x_0, x_1 uniformly and setting

$$x_2 = x - x_0 - x_1.$$

For P_0 the pair (x_0, x_1) is uniform. For P_1 the pair (x_1, x_2) is uniform, because x_0 is uniform and appears only in x_2. The same holds for P_2 with (x_2, x_0). In every case P_i's two shares are uniform on $\mathbb{Z}_{2^{64}}^2$ and independent of x. Shares of different values are independent. The simulator draws them uniformly.

Step 2: what the protocol does between messages. Every operation of both auctions is built from four interactive primitives: multiplication, AND, the reshare step of the shuffle, and opening. That covers comparison, the scans and validation. Everything else is local arithmetic on P_i's own shares and on values it has already received. So P_i's view is determined by its input shares, its keys, and the sequence of received messages. It is enough to show that each received message has, given everything P_i saw before it, a distribution the simulator can sample from pub and the simulated view so far.

The lengths of all messages, and which primitive runs when, depend only on pub. The code branches on the party index, the mode, the batch size and opened public values, never on a secret. The shape test (Section 4) checks this on every run.

Step 3: multiplication messages. P_{i+1} sends P_i, for each product,

$$z_{i+1} = x_{i+1}y_{i+1} + x_{i+1}y_{i+2} + x_{i+2}y_{i+1} + \alpha_{i+1},$$

with the zero-sharing mask

$$\alpha_{i+1} = F(k_{i+1})[w] - F(k_{i+2})[w'].$$

In H_1 the word $F(k_{i+2})[w']$ is uniform and used nowhere else, so

$$z_{i+1} \text{ is uniform and independent of everything } P_i \text{ has seen.}$$

The AND gate is the same over $\mathbb{Z}_2^{64}$ with XOR in place of $+$ and $-$. The simulator answers every multiplication and AND message with fresh uniform words.

Step 4: the shuffle. A shuffle applies $\pi = \pi_2 \circ \pi_1 \circ \pi_0$ in three reshare steps. In step c, the pair P_{c+1}, P_{c+2} permutes by π_c and reshares to the excluded server P_c. With r, m drawn from the pair's stream (key k_{c+2}), the new summands are

$$y_{c+2} = r, \qquad y_{c+1} = a' - r + m, \qquad y_c = b' - m.$$

P_i receives messages only in its own step, $c = i$, whose stream is that of k_{i+2}. It receives the pair

$$(y_{i+1},\, y_i) = (a' - r + m,\; b' - m).$$

In H_1, m is fresh and uniform, so y_i is uniform. Given y_i, the fresh uniform r makes y_{i+1} uniform. So

$$(y_{i+1}, y_i) \text{ is uniform on } \mathbb{Z}_{2^{64}}^2 \text{ and independent of everything } P_i \text{ has seen.}$$

In the other two steps P_i only sends. The simulator answers its own step with fresh uniform words.

This also shows something needed next. The piece π_i is drawn from the stream of k_{i+2}, and the only messages P_i receives that depend on it are masked by fresh r, m. Every message after the shuffle is masked afresh (Steps 3 and 4). So

$$\pi_i \text{ is uniform on } S_n \text{ and independent of } P_i\text{'s view up to the next opening,}$$

up to the sampler's bias. Fisher-Yates with a 64-bit word reduced modulo $j+1$ has bias at most $(j+1)/2^{64}$ at step j, so the total distance from uniform is at most $\sum_j (j+1)/2^{64} \le n^2/2^{65}$.

Step 5: permutation openings. To apply a secret permutation ρ, the servers shuffle ρ by a fresh π and open the result. The opened value is

$$\sigma = \rho \circ \pi^{-1} = \rho \circ \pi_0^{-1} \circ \pi_1^{-1} \circ \pi_2^{-1}.$$

Fix every value other than π_i. Writing the product as $A \circ \pi_i^{-1} \circ B$, the map

$$\tau \mapsto A \circ \tau^{-1} \circ B$$

is a bijection of S_n. A uniform π_i, independent of A, B and the view (Step 4), therefore gives

$$\sigma \text{ uniform on } S_n \text{ and independent of } P_i\text{'s view before the opening.}$$

The opening message is the missing summand, which is fixed by σ and P_i's own shares:

$$x_{i+2} = \sigma - x_i - x_{i+1} \quad \text{(coordinatewise).}$$

Each shuffle draws a fresh π, so the next opening has its own fresh π_i, and the argument repeats. The simulator draws σ uniformly from S_n and sends the summand that matches it.

Step 6: the remaining messages.

- *Validation* (when switched on) first sends each server its predecessor's copy of the summand it also holds, which for an honest trader equals its own s_0. It then exchanges per-order flags, all 0, and opens values that are public constants for a valid order: $b - b^2 = 0$, high bits $= 0$, validity $= 1$. The simulator writes those constants.
- *Every other opening* in a semi-honest run is a permutation opening. In particular no fill, price, volume or comparison result is ever opened to a server.

- *The exit barrier* in the server binary sends empty messages.

Step 7: the simulator. $\mathcal{S}_i$ draws uniform input shares (Step 1) and runs P_i's own code. It answers each multiplication, AND and reshare message with fresh uniform words (Steps 3 and 4). It answers each permutation opening with the summand for a fresh uniform $\sigma \in S_n$ (Step 5), and each validation message with its public constant (Step 6). It outputs the resulting view.

Step 8: conclusion. In H_1, every item P_i receives has, given the view before it, exactly the conditional distribution $\mathcal{S}_i$ uses, apart from the sampler bias in Step 5. P_i's own computation is the same function in both. By the chain rule the two joint distributions are equal up to $S \cdot n^2/2^{65}$. Adding the $H_0 \to H_1$ step gives the bound in the theorem. ■

A. 4. The tests (`crates/auction/tests/audit.rs`)

They check the two facts the proof relies on, on the actual code.

Shape, exact. Two batches of four orders are run: one where everything crosses on one symbol, one spread over symbols with different sides, sizes, minimums and limits. For each, every server's received message lengths and opening lengths are recorded. They must be identical, for Stage 1 and Stage 2, semi-honest and malicious. This is Step 2's "depends only on `pub`", checked directly.

Openings, statistical. Each batch is run 300 times with fresh randomness, and every opened value is recorded. For each opening, the two batches' distributions must be the same: a two-sample chi-squared test on a 16-bucket hash of the opened vector, at $p = 10^{-9}$. Every permutation opening must also be uniform over the 24 permutations of four elements. This is Step 5, checked directly, and Step 6's "constants stay constant", since a constant opening that changed with the data would split the buckets. Malicious mode is included: its extra openings (masked differences, zero checks, joint coins, the MAC check) must also not depend on the orders.

Planted leaks. Both keep every fill and price correct, so the differential tests cannot see them. Only the audit can.

- `plant-open-unshuffled` opens ρ without shuffling it first, so the sorting permutation, which is the order of the data, becomes public. Caught in Stage 1, Stage 2 and malicious mode, with a statistic of 600, the maximum for 300 runs per batch.
- `plant-stage2-debug-open` leaves a stray opening of the clearing prices in Stage 2, the kind of thing a debugging session leaves behind. Caught in both modes, with a statistic of 600.

With them, `check_planted.sh` runs 27 planted bugs.

What the tests cannot show. A chi-squared test on a hash bucket detects gross dependence, not a subtle bias. Four orders is a small batch. Passing is evidence that the implementation follows the protocol the proof is about, not a proof in itself.

A. 5. Scope: what this does not cover

- **Malicious servers.** In an honest run of malicious mode, every extra opening is a value masked by a fresh random triple, a check that is 0, a joint coin, a validation constant, or the MAC check, which is 0. So the view argument above extends to a server that follows the protocol, and the tests cover it. Against a server that deviates, the proof must also show that any deviation is caught before an opening it could influence. That is the job of the FLNW product checks and the MAC-checked shuffle (`docs/design/malicious.md`). The full simulation proof, including the two ring extensions, is Milestone 6 work.
- **Malicious traders.** Validation opens, for an invalid order, a value that depends on that order (for example $b - b^2$ for a bad bit). That reveals something about the cheating trader's own input, to every server, and nothing about anyone else.
- **Side channels.** Message sizes and counts are fixed by `pub`, which the shape test checks. CPU timing and memory access patterns have not been audited. Local work on secrets uses fixed-time integer arithmetic in Rust, but nothing here proves constant time.
- **Key setup.** The theorem assumes each PRF key is known to exactly its two servers. The keys are agreed over mutually authenticated TLS at setup, which is assumed correct.
- **What traders learn.** The proof is about servers. Traders learn what the auction itself reveals: their fills and, in Stage 2, the clearing price. That includes the inherent leaks in `threat-model.md`: probing ("pinging"), and the fact that the price is always some order's limit.

Appendix B: benchmark report

All numbers here were measured on real machines: three servers on three Google Cloud VMs, talking over the VPC's internal network through the same mutually authenticated TLS used everywhere else. Nothing is simulated. Every run's output was reconstructed and compared with the plain reference matcher before it was recorded; no run in the tables failed that check. Raw data: `results/gcp/runs.txt`, network measurements in `results/gcp/net-*.txt`. Produced by `scripts/gcp_bench.sh` at commits 832fe01 (LAN) and 9600ae1 (WAN), which contain the same protocol code.

B. Setup

Machines. `n2-standard-2`: 2 vCPUs (one physical core with two hyperthreads) and 8 GB per server. Larger machines did not fit the free-trial quota (8 C3 vCPUs per region, no increases allowed during the trial), so these numbers are a floor for the hardware, not a ceiling.

Networks, as measured (`ping`, 20 packets; `iperf3`, one TCP stream for 5 s):

Profile	Placement	Round trip between servers	Bandwidth
LAN	all three in us-west1-b	0.19 to 0.21 ms average	9.27 Gbit/s
WAN	us-west1-b, us-central1-a, us-east4-b	41.9, 27.3 and 68.5 ms	0.745 Gbit/s

Workload. Random batches over 8,192 hidden symbols (13-bit symbol field), sizes 1 to 10,000. Stage 1 orders have random minimum fills and run 3 elimination passes. Stage 2 limits fall within 50 ticks of 10,000 so books cross. The protocols are oblivious: their cost does not depend on the orders, only on the batch size and parameters, so the choice of workload does not affect the timings.

Measured quantity. Wall-clock time of the auction on the slowest server, from the first protocol message to the final share of each fill (TLS setup and key agreement excluded; they happen once per deployment). Median of 3 runs. Malicious mode includes input validation, all product and shuffle checks and the unanimous verdict. Data is the most bytes sent by any one server.

B. Results

Time (median of 3)

Orders	Stage 1 semi-honest	Stage 1 malicious	Stage 2 semi-honest	Stage 2 malicious
LAN				
100	0.020 s	0.130 s	0.029 s	0.202 s
1,000	0.055 s	0.948 s	0.108 s	1.451 s
10,000	0.359 s	9.878 s	0.894 s	15.697 s

Orders	Stage 1 semi-honest	Stage 1 malicious	Stage 2 semi-honest	Stage 2 malicious
100,000	4.328 s		12.620 s	
WAN				
100	5.978 s	14.137 s	6.510 s	21.402 s
1,000	7.329 s	17.085 s	8.222 s	25.623 s
10,000	7.932 s	44.230 s	10.640 s	82.048 s
100,000	23.626 s		63.563 s	

Malicious mode was run up to 10,000 orders. At 1,000 orders and up, the 3 runs of a configuration were within about 7% of each other on the LAN. On the WAN, three runs were outliers (15.5 s against 7.7 to 7.9 s, 61.8 s against 42 to 44 s, 118 s against 79 to 82 s), which the median absorbs. The Stage 2 semi-honest 1,000-order cell has a fourth run, the smoke test.

Throughput (orders per second, from the medians)

Orders	Stage 1 semi	Stage 1 malicious	Stage 2 semi	Stage 2 malicious
LAN 1,000	18,182	1,055	9,259	689
LAN 10,000	27,855	1,012	11,186	637
LAN 100,000	23,105		7,924	
WAN 1,000	136	59	122	39
WAN 10,000	1,261	226	940	122
WAN 100,000	4,233		1,573	

Cost per auction

Orders	Stage 1 semi	Stage 1 malicious	Stage 2 semi	Stage 2 malicious
Rounds at 1,000	280	566	297	855
Rounds at 10,000	296	729	313	1,081
Data per server, per order	7 to 8 KiB	158 to 171 KiB	16 to 20 KiB	252 to 278 KiB

Statistical security of the malicious runs, from each run's own accounting (FLNW bound per verified batch, union bound over batches and shuffle checks): Stage 1 at least 45.7, 45.6 and 44.6 bits at 100, 1,000 and 10,000 orders; Stage 2 at least 44.7, 44.6 and 43.9 bits. It falls slowly with batch size because more verification batches enter the union bound.

Rounds grow only logarithmically with the batch: from 100 to 100,000 orders, Stage 1 semi-honest goes from 268 to 308. Data grows linearly.

B. What the numbers show

On a LAN, malicious security costs about 13 to 28 times semi-honest time (Stage 1: 17 times at 1,000 orders, 28 at 10,000; Stage 2: 13 and 18). Malicious mode sends 15 to 23 times the data, and most of it is the Milestone 4a product checks:

every multiplication is checked against a bucket of random triples, each of which must itself be produced. The FLNW bound sets the bucket size from how many products are verified together: 4 for a full chunk of 2^{18}, more for smaller batches.

On a WAN, rounds set a floor. Each round costs about 22 to 30 ms, close to half the slowest link's 68.5 ms round trip, because most rounds pass a message one hop around the ring of servers. So any Stage 1 auction takes at least about 6 s and any malicious one at least about 14 s, however small. Larger batches amortise that floor: semi-honest Stage 1 goes from 17 orders/s at 100 orders to 4,233 at 100,000. A WAN deployment should run large, infrequent batches.

At 100,000 orders, bandwidth takes over. Stage 2 sends 2 GB per server. On the WAN that run took 64 s against 9 s of pure latency, so most of the time went to moving data at roughly 0.3 Gbit/s per server, below the measured single-stream 0.745 Gbit/s because each server feeds two links and the 68 ms link limits each TCP window.

Stage 2 costs about 2.5 times Stage 1's data but nearly the same number of rounds, so on the WAN it is only 1.1 to 1.5 times slower at up to 10,000 orders semi-honest.

B. Against prior work

The only published comparable system is Cartlidge, Smart and Talibi Alaoui, "MPC Joins The Dark Side" (AsiaCCS 2019, ePrint 2018/1045). Their three-party volume match with malicious security, N = M = 500 (1,000 orders), Table 8; numbers checked against the paper, see `docs/prior-work.md`:

	2018/1045, Table 8 (3 parties)	darkmpc Stage 1, malicious
Online time	0.900 s	0.948 s (no separate offline phase)
Offline (preprocessing)	4.136 s	none
End to end	5.036 s	0.948 s
End-to-end throughput	about 200 orders/s	1,055 orders/s
Rounds	4,509 (9(max(N,M)+1))	566
Largest batch reported	1,000	10,000 (1,012 orders/s)
Network	LAN, 0.47 ms ping, 10 Gbit/s	LAN, 0.2 ms round trip, 9.3 Gbit/s
Machines	i7-7700K (4 cores), 32 GB	1 physical core (2 vCPUs), 8 GB
Statistical security	40 bits	at least 45.6 bits (computed per run)

What this comparison supports. On a LAN, darkmpc's end-to-end time for an actively secure 1,000-order batch is about 5 times lower, with 8 times fewer rounds, and it keeps the same throughput at 10 times the batch size.

What it does not support. It compares two systems, not one algorithm against another:

- **Mechanism.** Their volume match handles one instrument and has no minimum fills. darkmpc's Stage 1 hides the symbol and side of every order across 8,192

instruments, validates traders' inputs on shares, and runs 3 minimum-fill elimination passes. That is strictly more work, so this direction of difference favours them.

- **Network.** Our LAN round trip is less than half theirs, which favours us. How much is hard to say: 4,509 strictly sequential rounds at 0.47 ms would take 2.1 s, more than their measured 0.9 s, so their round count cannot be a count of sequential round trips (0.9 s over 4,509 rounds is 0.2 ms each, under one 0.235 ms one-way trip). The paper does not say how its rounds map to latency.
- **Hardware.** Our servers have one physical core against their four, but our cores are newer. How much of the 5 times comes from the protocol and how much from the hardware is not separable from these numbers.
- **Software.** They used SCALE-MAMBA, a general-purpose framework over a 128-bit prime field. darkmpc is purpose-built over the 64-bit ring.

WAN. Neither 2018/1045 nor 2020/662 measured a WAN, so there is no measured number to compare. An estimate, not a measurement: their LAN figures work out to 0.2 ms per round on a 0.47 ms round trip, about 0.43 of a round trip each. darkmpc's malicious rounds on this WAN cost about 26 to 30 ms on a 68.5 ms slowest round trip, about 0.38 to 0.44 of one. If their rounds scaled the same way, 4,509 of them would take on the order of 2 minutes on this WAN for one 1,000-order batch, against darkmpc's measured 17.1 s. That assumes their rounds behave like ours, which is unverified, so it belongs in the write-up only as a labelled extrapolation.

Never compare darkmpc's semi-honest numbers with their malicious ones.

B. Reproducing

```
scripts/gcp_bench.sh quota lan
scripts/gcp_bench.sh up lan && scripts/gcp_bench.sh deploy lan && scripts/gcp_bench.sh net lan
scripts/gcp_bench.sh grid lan
scripts/gcp_bench.sh down lan
```

For the WAN, `export WAN_ZONES="us-west1-b us-central1-a us-east4-b"` first (us-east4-a had no `n2-standard-2` capacity at the time) and use `wan`. The LAN grid took about 20 minutes and the WAN grid about 50, most of it uploading dealt orders from a laptop on campus wifi.